

Permisos, modelos y construir con Claude Code

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 3 de 12 · Notas en español (es-ES)

El último bloque didáctico del 6 de agosto fue cómo **governáis** Claude Code antes de dejarle escribir una app: modos de permiso, `/model`, `/config`, dónde viven los archivos de ajustes y `/init` → **CLAUDE.md** como ley persistente del proyecto. Módulos posteriores profundizan en modos y agentes; este capítulo es el panel de control Foundation.

Modos de permiso (según se etiquetaron en clase)

La UI de Code agrupaba con qué libertad el agente puede usar herramientas:

Modo (lenguaje de aula)	Qué estáis eligiendo	Cuándo lo usaría Vijay
Preguntar / permiso manual por herramienta	Cada llamada sensible espera	Repo no fiable, primer día en código de cliente
Auto-editar en el espacio de trabajo	Las ediciones siguen sin un clic cada vez	Una carpeta que ya confiastéis al arrancar el CLI
Planificar primero	Emitir un plan en secuencia; revisáis/personalizáis antes de ejecutar	Brownfield, refactors caros, estilo examen «no ejecutes a ciegas»
Bypass / no esperar	Saltar avisos de permiso y correr	Solo en sandboxes de usar y tirar — no en árboles de cliente UST

El primer arranque del CLI ya preguntó «¿confiáis en esta carpeta git?». Ese consentimiento es la raíz del auto-edit. Las listas allow/deny también persisten en el JSON de ajustes (denegar la lectura de secretos/ claves de API fue el ejemplo de Lalit). Podéis pedir a Claude que reescriba sus propios ajustes — igual necesita permiso para tocar esos archivos.

Esfuerzo y `/model`

El esfuerzo es cuánto debe pensar el modelo seleccionado—no un producto aparte. `/model` listó lo que habilitaba la suscripción de la org (en clase: Sonnet, Opus, Haiku, más nombres como Fable). Escribir `opus` tomaba el SKU Opus más reciente que tenía (citó una línea clase 4.8); también podéis pasar un ID completo de los docs. Pasar a un Sonnet de 1M cambia la *ventana*, no automáticamente la calidad de una respuesta de una línea. Pregunta de Hussain: no hace falta memorizar cada ID con fecha si el nombre corto resuelve al vigente.

SELECTOR FRENTE A DOCUMENTACIÓN

Sonnet «5», Opus «4.8», Fable, 272K frente a 1M eran UI del 6 de agosto de 2026. Las respuestas escritas del examen deben usar IDs y capacidades de **la página de modelos**.

Slash config y capas de settings.json

`/config` muestra pares clave/valor (backend del editor: Git Bash frente a PowerShell frente a vim; idioma de la UI; interruptores de agente; auto-compact). Los valores se guardan bajo un directorio `.claude` (ASR: «dot clod») como `settings.json` en más de una capa:

1. **Nivel de usuario** — bajo el perfil del sistema operativo; aplica a todos los proyectos de esa persona. Crear auto-compact desde `/config` suele aterrizar aquí primero.
2. **Nivel de proyecto** — `.claude/settings.json` en el repo git (convenios compartibles).
3. **Override de carpeta / local** — `settings.local.json` para anular ajustes de proyecto en esta máquina sin commitear secretos.

Precedencia que enseñó: si existen los tres, **gana carpeta/local**; si no, proyecto; si no, usuario. Las claves que faltan se heredan de la capa de encima. Este JSON son los interruptores de Claude Code—no vuestras reglas ESLint. Los estándares de código limpio, pines de bibliotecas y «nunca leas `.env`» como *comportamiento* van en **CLAUDE.md** (memoria/instrucciones), que es un archivo distinto de `settings.json`.

CLAUDE.md vía /init

`/init` intenta generar instrucciones persistentes de proyecto. En un árbol vacío o poco claro puede negarse hasta que haya estructura—podéis crear el markdown vosotros. Es la misma familia de archivo que Cowork auto-creó en la sesión 1. Las reglas de base (solo OpenAPI para el frontend, nada de PII de cliente en prompts, gestor de paquetes) van aquí para que cada sesión no reexplique el repo. No pegéis manuales enteros del fabricante (aviso de tokens de la sesión 1).

Construir: igual debéis un prompt y una revisión

Con permisos, modelo y CLAUDE.md en su sitio, podéis pedir una app de extremo a extremo. Promesa de Vijay: Code planificará, usará herramientas y verificará. Vuestro trabajo en CCDV-F es saber *qué mandos habéis girado* y qué sigue necesitando a un humano (secretos, despliegue a producción, datos de cliente). Sesiones posteriores generan sin teclear la app a mano; el examen igual preguntará por qué elegisteis modo plan, un modelo más pequeño o una sesión nueva.

Lista antes de «generar y ya»

- El diálogo de confianza y el modo de permiso coinciden con la sensibilidad del árbol.
- `/model` encaja coste frente a necesidad de ventana.
- Auto-compact activado si esperáis un bucle agentic largo.
- CLAUDE.md declara pila y prohibiciones; `settings.json` declara interruptores de producto.
- Sesión nueva para una funcionalidad nueva para que el experimento fallido de ayer no esté en contexto.

Documentación oficial

- **Resumen de Claude Code**

<https://code.claude.com/docs/en/overview>

Permisos, CLI, IDE, CLAUDE.md según se documentan ahora.

- **Modelos**

<https://docs.claude.com/en/docs/about-claude/models>

IDs para pasar a /model.

- **Pensamiento extendido**

<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>

A qué se mapea «esfuerzo / thinking» en el mundo API.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Contraparte empresarial de las listas deny en settings.json.

- **Política de uso**

<https://www.anthropic.com/legal/usage-policy>

Lo que nunca debe aterrizar en un prompt de un espacio de trabajo de confianza.

- **Salidas estructuradas**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

En la lista de competencias de Developer Foundation junto a Code.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.